

Modul 9 Syscall Xinu

Tujuan Praktikum

1. Praktikan mampu memahami syscall pada Xinu.
2. Praktikan mampu mengimplementasikan syscall pada Xinu

9.1 Definisi dan Fungsi Syscall

Syscall didefinisikan sebagai interface layanan yang diberikan oleh sistem operasi. Proses akan menggunakan syscall jika ingin mendapatkan layanan dari os. Selain itu, syscall juga memberikan lapisan perlindungan keamanan dan information hiding. Proses tidak perlu mengetahui detail internal syscall. Proses cukup memanggil syscall yang diinginkan, kernel os akan mengeksekusinya dan proses menerima hasilnya (berhasil atau gagal syscall tersebut). Syscall berbeda dengan fungsi yang dibuat oleh developer misal fungsi printf().

System calls, which define operating system services for applications, protect the system from illegal use and hide information about the underlying implementation.

Gambar 9-1 Definisi dan Fungsi Syscall

9.2 Cara Kerja Syscall Pada Xinu

- a. Disable semua interupsi pada saat syscall dipanggil

Hal ini dilakukan untuk memastikan bahwa perubahan struktur data global yang dilakukan pada saat syscall konsisten untuk seluruh os. Syscall harus menjamin bahwa tidak ada proses lain yang mengubah struktur data os pada saat yang bersamaan atau dapat menghasilkan struktur data yang tidak konsisten.

Selain untuk memastikan struktur data konsisten secara global, disable interupsi juga mencegah terjadinya context switching. Jangan sampai terjadi context switching ke proses lain pada saat syscall sedang dijalankan.

System calls must disable interrupts to prevent other processes from changing global data structures; using a disable / restore paradigm increases generality.

Gambar 9-2 Disable Interrupts Syscall

- b. Memeriksa seluruh argument yang diberikan oleh proses(caller).

Syscall tidak boleh membuat asumsi apapun mengenai proses yang memanggil. Syscall akan memeriksa semua argument dan ijin yang ada. Jika terdapat argument yang tidak sesuai atau tidak mempunyai ijin maka syscall akan menolak permintaan syscall oleh proses. Contoh: syscall sleep() hanya menerima argument integer positif. Jika argument adalah string atau bilangan negatif maka akan ditolak.

- c. Mengesekusi layanan yang diminta oleh process (caller). Inilah tugas utama pada syscall. Contoh: jika syscall adalah freemem() maka fungsi utama adalah membebaskan memori block pada alamat tertentu.
- d. Restore interupsi sehingga proses lain bisa menggunakan syscall/context switch.
- e. Memberikan laporan hasil (sukses/gagal) kepada proses (caller).

9.3 Template Syscall Xinu

Untuk menjamin semua syscall berjalan dengan baik maka semua syscall Xinu menggunakan template berikut ini:

```
syscall function_name ( args ) {
    intmask mask;                /* Saved interrupt mask          */
    mask = disable();            /* Disable interrupts at start of function */
    if ( args are incorrect ) {
        restore(mask);          /* Restore interrupts before error return */
        return SYSERR;
    }
    ... other processing ...
    if ( an error occurs ) {
        restore(mask);          /* Restore interrupts before error return */
        return SYSERR;
    }
    ... more processing ...
    restore(mask);               /* Restore interrupts before normal return */
    return appropriate value ;
}
```

Gambar 9-3 Template Syscall Xinu

9.4 Syscall Resume

Berikut adalah contoh kode syscall resume. Syscall resume() akan mengubah status proses dari state suspend ke state ready dan akan mengembalikan (return value) prioritas proses ready tersebut.

```

5  /*-----
6  * resume - Unsuspend a process, making it ready
7  *-----
8  */
9  pri16 resume(
10     pid32 pid /* ID of process to unsuspend */
11 )
12 {
13     intmask mask; /* Saved interrupt mask */
14     struct proctent *prptr; /* Ptr to process's table entry */
15     pri16 prio; /* Priority to return */
16
17     mask = disable(); 1
18     if (isbadpid(pid)) {
19         restore(mask); 2
20         return (pri16)SYSERR;
21     }
22     prptr = &proctab[pid];
23     if (prptr->prstate != PR_SUSP) { 3
24         restore(mask);
25         return (pri16)SYSERR;
26     }
27     prio = prptr->prprio; /* Record priority to return */
28     ready(pid); 4
29     restore(mask); 5
30     return prio; 6
31 }

```

Gambar 9-4 Kode Syscall resume()

Menggunakan template 7-3 yang diberikan berikut adalah penjelasan detail cara kerja syscall resume: Setelah mendeklarasikan variabel-variabel yang akan digunakan (baris 13-15),

- [1] syscall disable interupsi (baris 17)
- [2] syscall memeriksa validitas argument yang diberikan . PID tidak boleh negatif, terdapat batas maksimum PID , dst. Jika PID tidak memenuhi syarat maka restore interupsi dan berikan hasil SYSERR.
- [3] dan [4] merupakan pekerjaan utama syscall. Pertama pastikan bahwa proses dalam status suspend (baris 23) jika proses tidak dalam state suspend maka berikan hasil SYSERR. Jika dalam state suspend maka ubah menjadi state ready (baris 28). Fungsi ready() adalah fungsi biasa bukan syscall.
- [5] dan [6] Enable interupsi dan berikan hasil syscall yaitu prioritas proses (prio).

9.5 Membuat Syscall Baru pada Xinu

Cara membuat Syscall baru pada Xinu:

- a. Buat file baru pada folder ./system
- b. Edit file tersebut dan pastikan mengikuti template code syscall Xinu.
- c. Edit ./include/prototypes.h dengan menambahkan nama fungsi yang dibuat.
- d. Compile Xinu dengan tambahan syscall baru dan bisa digunakan.

Contoh:

Kita akan membuat dummy syscall bernama chname(). Syscall ini tidak melakukan apapun hanya menampilkan kalimat "New Syscall is easy". Template diambil dari chprio.c

- a. \$ touch ./system/chname.c
- b. Berikut isi dari chname.c

```

/* chname.c - chname */

#include <xinu.h>

/*-----
 * chname - Dummy syscall
 *-----
 */
pri16 chname(
    pid32      pid,          /* ID of process to change */
    pri16      newprio       /* New priority */
)
{
    intmask mask;           /* Saved interrupt mask */
    struct procent *prptr;   /* Ptr to process's table entry */
    pri16      oldprio;      /* Priority to return */

    mask = disable();
    if (isbadpid(pid)) {
        restore(mask);
        return (pri16) SYSERR;
    }
    prptr = &proctab[pid];
    oldprio = prptr->prprio;
    prptr->prprio = newprio;
    kprintf("\n\nNew Syscall is easy\n\n");
    restore(mask);
    return oldprio;
}

```

Gambar 9-5 Contoh Kode Syscall Baru Pada Xinu

Pada dasarnya sama dengan chprio.c, hanya ditambahkan kprintf() sebelum restore(mask).

- c. Edit ./include/prototypes.h

```

/* in file ascdatetime.c */
extern status ascdatetime(uint32, char *);

/* in file bufinit.c */
extern status bufinit(void);

/* in file chprio.c */
extern pri16 chprio(pid32, pri16);

/* in file chname.c */
extern pri16 chname(pid32, pri16); 1

/* in file clkupdate.S */
extern uint32 clkcount(void);

/* in file clkhandler.c */
extern interrupt clkhandler(void);

```

Gambar 9-6 Kode File Prototypes.h

Kata kunci extern berarti fungsi tersebut (syscall) dapat diakses secara global oleh semua fungsi lainnya.

pri16 adalah return value syscall/fungsi.

chname(pid32, pri16) adalah nama fungsi (syscall) beserta parameternya.

- d. Compile
 - \$ make clean
 - \$ make

9.6 Cara Penggunaan Syscall

Setelah syscall dibuat maka syscall sudah bisa digunakan. Cara menggunakan syscall cukup dengan langsung memanggil syscall tersebut.

Contoh: Kita akan edit xsh_uptime.c dan menambahkan syscall chname() sehingga setiap kali perintah uptime dieksekusi syscall chname() juga akan dieksekusi.

- a. Buka ./shell/xsh_uptime.c
- b. Edit hingga isinya xsh_uptime.c seperti ini

```
printf("Xinu has been up ");
if (days > 0) {
    printf(" %d day(s) ", days);
}

if (hrs > 0) {
    printf(" %d hour(s) ", hrs);
}

if (mins > 0) {
    printf(" %d minute(s) ", mins);
}

if (secs > 0) {
    printf(" %d second(s) ", secs);
}
printf("\n");
chname(1,20);
return 0;
```

Gambar 9-7 Contoh Penggunaan Syscall Pada Perintah Uptime

- c. Compile Xinu